01 / PROBLEM AND CONTRIBUTION

Five-stage pipelined MIPS processor

Advanced Digital Design & Verification · Lab 2 | Fall 2026 | Two-person team

RESULT AT A GLANCE

Reduced a six-instruction directed sequence from 16 to 10 cycles with forwarding-a 37.5% cycle-count reduction for that sequence-and retained the eight-instruction base regression.

The engineering question

How can a baseline processor be pipelined while preserving correct execution through data and branch hazards?

Situation

Overlapping instruction execution introduces dependencies and wrong-path instructions. The project extends a provided processor into a five-stage pipeline while retaining correct instruction behavior.

Task

Develop forwarding, load-use stalls, and BEQ flushing, and verify these mechanisms with directed tests and the base instruction regression.

My contribution

Worked on the five-stage pipeline, forwarding, hazard detection, BEQ flushing, ADD/LW/BEQ tests, and synthesis/debug review with Akshith Alwala.

Tools and concepts

SystemVerilog / Pipelining / Hazard detection / Design Compiler / VCS / Verdi

02 / DESIGN AND METHOD

From the task to implementation

01 / Implementation step

Partitioned the provided non-pipelined processor into IF, ID, EX, MEM, and WB, carrying control and operand data through four register boundaries.

02 / Implementation step

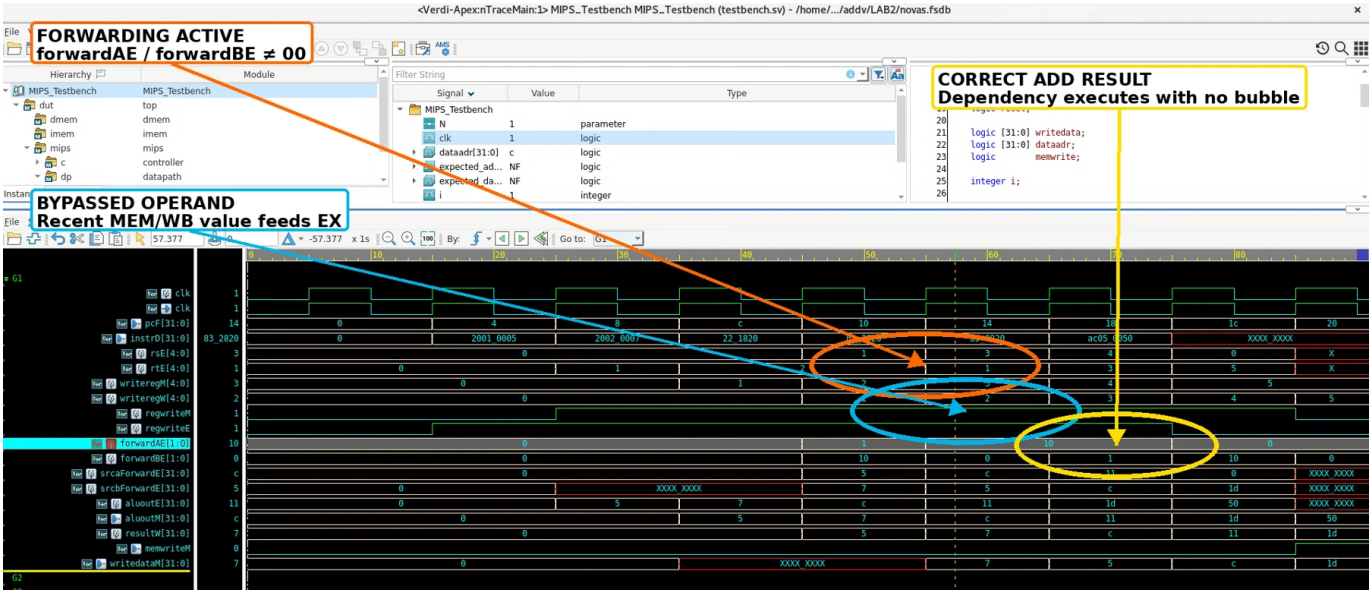
Added operand forwarding, load-use stalls, and taken-BEQ flushing. Directed waveforms trace arithmetic dependencies, load writeback, and wrong-path invalidation.

03 / Implementation step

The team extended the core with MULADD and PERFMON and reran the original instruction regression. A synthesis reset issue was resolved by handling flush separately from asynchronous reset.

Hazards in the pipeline

Forwarding routes available results to dependent instructions. A load-use dependency can still require a stall, and a taken BEQ requires the wrong-path instructions to be invalidated. Each mechanism is checked with directed traces.



Forwarding during an ADD dependency

Annotated simulation waveform showing the operand forwarding path.

03 / RESULTS AND ENGINEERING PERSPECTIVE

Evidence, interpretation, and next steps

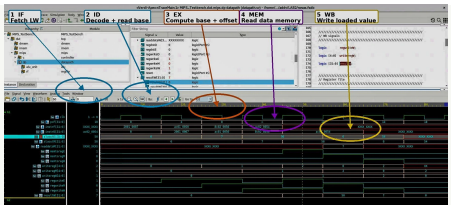
Pipeline stages	5
Directed sequence	10 vs 16 cycles
Base instructions	8 checked
Directed cycle reduction	37.5%

Reported result

In the six-instruction directed forwarding example, execution used 10 cycles with forwarding versus 16 without it. The original ADD, SUB, AND, OR, SLT, LW, SW, and BEQ regression passed after the extensions.

Cycles versus clock period

The 37.5% reduction is calculated as (16 - 10) / 16 for the six-instruction example. Fewer cycles do not automatically imply a faster synthesized clock or lower elapsed time; the report documents a longer critical path in its pipelined comparison.



Load-use hazard handling

Annotated waveform from the pipeline hazard tests.

Scope and limitations

The 10-versus-16 comparison is specific to that directed sequence. The report's pipelined core had a longer synthesized critical path than its single-cycle comparison, so pipelining is not presented as a demonstrated clock-frequency gain.

Possible next steps - proposed, not completed results

- Report cycle count and synthesized clock period together for a defined program set.
- Expand dependency, stall, and branch-interaction regressions beyond the demonstrated directed sequence.

Source material: Team Lab 2 report, pp. 3-8, 14-16.

Prepared from the supplied reports, project files, and verified portfolio copy. Figures retain their original content; explanatory diagrams are labelled.